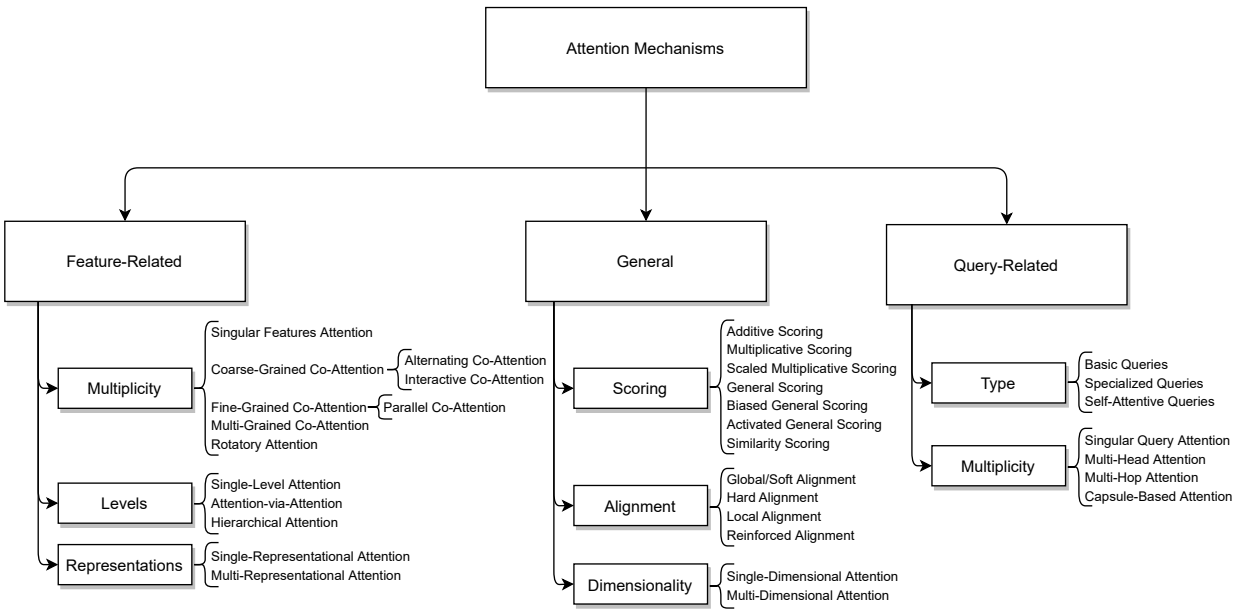

Fig. 3. A taxonomy of attention mechanisms.

TABLE 1
Notation.

Symbol	Description
$\boldsymbol{F}$	Matrix of size $d_f \times n_f$ containing the feature vectors $\boldsymbol{f}_1, \ldots, \boldsymbol{f}_{n_f} \in \mathbb{R}^{d_f}$ as columns. These feature vectors are extracted by the feature model.
$\boldsymbol{K}$	Matrix of size $d_k \times n_f$ containing the key vectors $\boldsymbol{k}_1, \ldots, \boldsymbol{k}_{n_f} \in \mathbb{R}^{d_k}$ as columns. These vectors are used to calculate the attention scores.
$\boldsymbol{V}$	Matrix of size $d_v \times n_f$ containing the value vectors $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_{n_f} \in \mathbb{R}^{d_v}$ as columns. These vectors are used to calculate the context vector.
$\boldsymbol{W}_K$	Weights matrix of size $d_k \times d_f$ used to create the $\boldsymbol{K}$ matrix from the $\boldsymbol{F}$ matrix.
$\boldsymbol{W}_V$	Weights matrix of size $d_v \times d_f$ used to create the $\boldsymbol{V}$ matrix from the $\boldsymbol{F}$ matrix.
$\boldsymbol{q}$	Query vector of size d_q. This vector essentially represents a question, and is used to calculate the attention scores.
$\boldsymbol{c}$	Context vector of size d_v. This vector is the output of the attention model.
$\boldsymbol{e}$	Score vector of size d_{n_f} containing the attention scores $e_1, \ldots, e_{n_f} \in \mathbb{R}^1$. These are used to calculate the attention weights.
$\boldsymbol{a}$	Attention weights vector of size d_{n_f} containing the attention weights $a_1, \ldots, a_{n_f} \in \mathbb{R}^1$. These are the weights used in the calculation of the context vector.

orthogonal dimensions of an attention model. An attention model can consist of a combination of techniques taken from any or all categories. Some characteristics, such as the scoring and alignment functions, are generally required for any attention model. Other mechanisms, such as multi-head attention or co-attention are not necessary in every situation. Lastly, in Table 1, an overview of used notation with corresponding descriptions is provided.

3.1 Feature-Related Attention Mechanisms

Based on a particular set of input data, a feature model extracts feature vectors so that the attention model can attend to these various vectors. These features may have specific structures that require special attention mechanisms to handle them. These mechanisms can be categorized to deal with one of the following feature characteristics: the multiplicity of features, the levels of features, or the representations of features.

3.1.1 Multiplicity of Features

For most tasks, a model only processes a single input, such as an image, a sentence, or an acoustic sequence. We refer to such a mechanism as **singular features attention**. Other models are designed to use attention based on multiple inputs to allow one to introduce more information into the model that can be exploited in various ways. However, this does imply the presence of multiple feature matrices that require special attention mechanisms to be fully used. For example, [32] introduces a concept named **co-attention** to allow the proposed visual question answering (VQA) model to jointly attend to both an image and a question.

Co-attention mechanisms can generally be split up into two groups [33]: **coarse-grained co-attention** and **fine-grained co-attention**. The difference between the two groups is the way attention scores are calculated based on the two feature matrices. Coarse-grained attention mechanisms use a compact representation of one feature matrix as a query when attending to the other feature vectors. Fine-grained co-attention, on the other hand, uses all feature vectors of one input as queries. As such, no information is lost, which is why these mechanisms are called fine-grained.

As an example of coarse-grained co-attention, [32] proposes an **alternating co-attention** mechanism that uses the context vector (which is a compact representation) from one attention module as the query for the other module, and vice versa. Alternating co-attention is presented in Fig. 4. Given a set of two input matrices $\boldsymbol{X}^{(1)}$ and $\boldsymbol{X}^{(2)}$, features are extracted by a feature model to produce the feature matrices $\boldsymbol{F}^{(1)} \in \mathbb{R}^{d_f^{(1)} \times n_f^{(1)}}$ and $\boldsymbol{F}^{(2)} \in \mathbb{R}^{d_f^{(2)} \times n_f^{(2)}}$, where $d_f^{(1)}$

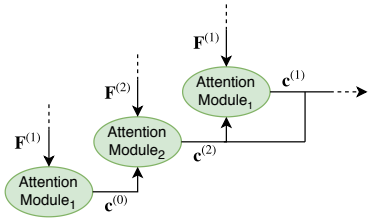

Fig. 4. An illustration of alternating co-attention.

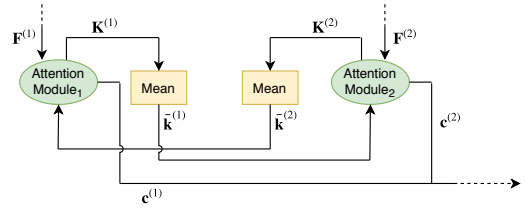

Fig. 5. An illustration of interactive co-attention.

and $d_f^{(2)}$ represent, respectively, the dimension of the feature vectors extracted from the first and second inputs, while $n_f^{(1)}$ and $n_f^{(2)}$ represent, respectively, the amount of feature vectors extracted from the first and second inputs. In [32], co-attention is used for VQA, so the two input matrices are the image data and the question data, for which the feature model for the image consists of a CNN model, and the feature model for the question consists of word embeddings, a convolutional layer, a pooling layer, and an LSTM model. Firstly, attention is calculated for the first set of features $\boldsymbol{F}^{(1)}$ without the use of a query (Attention Module$_1$ in Fig. 4). In [32], an adjusted additive attention score function is used for this attention mechanism. The general form of the regular additive score function can be seen in (6).

$$\text{score}(\underset{d_q\times 1}{\boldsymbol{q}}\ ,\ \underset{d_k\times 1}{\boldsymbol{k}_l}) = \underset{1\times d_w}{\boldsymbol{w}^T}\times \text{act}(\underset{d_w\times d_q}{\boldsymbol{W}_1}\times \underset{d_q\times 1}{\boldsymbol{q}} + \underset{d_w\times d_k}{\boldsymbol{W}_2}\times \underset{d_k\times 1}{\boldsymbol{k}_l} + \underset{d_w\times 1}{\boldsymbol{b}}), \quad (6)$$

where act() is a non-linear activation function, and $\boldsymbol{w} \in \mathbb{R}^{d_w}$, $\boldsymbol{W}_1 \in \mathbb{R}^{d_w\times d_q}$, $\boldsymbol{W}_2 \in \mathbb{R}^{d_w\times d_k}$, and $\boldsymbol{b} \in \mathbb{R}^{d_w}$ are trainable weights matrices, for which d_w is a predefined dimension of the weight matrices. A variant of this score function adapted to be calculated without a query for the application at hand can be seen in (7).

$$\underset{1\times 1}{e_l^{(0)}} = \underset{1\times d_w}{\boldsymbol{w}^{(1)T}} \times \text{act}(\underset{d_w\times d_k^{(1)}}{\boldsymbol{W}^{(1)}}\times \underset{d_k^{(1)}\times 1}{\boldsymbol{k}_l^{(1)}} + \underset{d_w\times 1}{\boldsymbol{b}^{(1)}}), \quad (7)$$

where $\boldsymbol{w}^{(1)} \in \mathbb{R}^{d_w}$, $\boldsymbol{W}^{(1)} \in \mathbb{R}^{d_w\times d_k^{(1)}}$, and $\boldsymbol{b}^{(1)} \in \mathbb{R}^{d_w}$ are trainable weight matrices for Attention Module$_1$, $\boldsymbol{k}_l^{(1)} \in \mathbb{R}^{d_k^{(1)}}$ is the lth column of the keys matrix $\boldsymbol{K}^{(1)}$ that was obtained from $\boldsymbol{F}^{(1)}$ via a linear transformation (see (1)), for which d_w is a prespecified dimension of the weight matrices and $d_k^{(1)}$ is a prespecified dimension of the key vectors.

Perhaps one may wonder why the query is absent when calculating attention in this manner. Essentially, the query in this attention model is learned alongside the other trainable parameters. As such, the query can be interpreted as a general question: "Which feature vectors contain the most important information?". This is also known as a **self-attentive mechanism**, since attention is calculated based only on the feature vectors themselves. Self-attention is explained in more detail in Subsection 3.3.1.

The scores are combined with an alignment function (see (3)), such as the softmax function, to create attention weights used to calculate the context vector $\boldsymbol{c}^{(0)} \in \mathbb{R}^{d_v^{(1)}}$ (see (4)). This context vector is not used as the output of the attention model, but rather as a query for calculating the context vector $\boldsymbol{c}^{(2)} \in \mathbb{R}^{d_v^{(2)}}$, based on the second feature matrix $\boldsymbol{F}^{(2)}$, where $d_v^{(2)}$ is the dimension of the value vectors obtained from $\boldsymbol{F}^{(2)}$ via a linear transformation (see (1)). For this module (Attention Module$_2$ in Fig. 4), attention scores are calculated using another score function with $\boldsymbol{c}_0$ as query input, as presented in (8). Any function can be used in this situation, but an additive function is used in [32].

$$\underset{1\times 1}{e_l^{(2)}} = \text{score}(\underset{d_v^{(1)}\times 1}{\boldsymbol{c}^{(0)}}\ ,\ \underset{d_k^{(2)}\times 1}{\boldsymbol{k}_l^{(2)}}). \quad (8)$$

These attention scores are then used to calculate attention weights using, for example, a softmax function as alignment function, after which the context vector $\boldsymbol{c}^{(2)}$ can be derived as a weighted average of the second set of value vectors. Finally, the context vector $\boldsymbol{c}^{(2)}$ is used as a query for the first attention module, which will produce the context vector $\boldsymbol{c}^{(1)}$ for the first feature matrix $\boldsymbol{F}^{(1)}$. Attention scores are calculated according to (9). In [32], the same function and weight matrices as seen in (7) are used, but with an added query making it the same as the general additive score function (see (6)). The rest of the attention calculation is similar as before.

$$\underset{1\times 1}{e_l^{(1)}} = \text{score}(\underset{d_v^{(2)}\times 1}{\boldsymbol{c}^{(2)}}\ ,\ \underset{d_k^{(1)}\times 1}{\boldsymbol{k}_l^{(1)}}). \quad (9)$$

The produced context vectors $\boldsymbol{c}^{(1)}$ and $\boldsymbol{c}^{(2)}$ are concatenated and used for prediction in the output model. Alternating co-attention inherently contains a form of sequentiality due to the fact that context vectors need to be calculated one after another. This may come with a computational disadvantage since it is not possible to parallelize. Instead of using a sequential mechanism like alternating co-attention, [34] proposes the **interactive co-attention** mechanism that can calculate attention on both feature matrices in parallel, as depicted in Fig. 5. Instead of using the context vectors as queries, unweighted averages of the key vectors are used as queries. The calculation of the average keys are provided in (10), and the calculation of the attention scores are shown in (11). Any score function can be used in this case, but an additive score function is used in [34].

$$\underset{d_k^{(1)}\times 1}{\bar{\boldsymbol{k}}^{(1)}} = \frac{1}{n_f^{(1)}}\sum_{l=1}^{n_f^{(1)}} \underset{d_k^{(1)}\times 1}{\boldsymbol{k}_l^{(1)}}, \qquad \underset{d_k^{(2)}\times 1}{\bar{\boldsymbol{k}}^{(2)}} = \frac{1}{n_f^{(2)}}\sum_{l=1}^{n_f^{(2)}} \underset{d_k^{(2)}\times 1}{\boldsymbol{k}_l^{(2)}}; \quad (10)$$

$$\underset{1\times 1}{e_l^{(1)}} = \text{score}(\underset{d_k^{(2)}\times 1}{\bar{\boldsymbol{k}}^{(2)}}\ ,\ \underset{d_k^{(1)}\times 1}{\boldsymbol{k}_l^{(1)}}),\quad \underset{1\times 1}{e_l^{(2)}} = \text{score}(\underset{d_k^{(1)}\times 1}{\bar{\boldsymbol{k}}^{(1)}}\ ,\ \underset{d_k^{(2)}\times 1}{\boldsymbol{k}_l^{(2)}}). \quad (11)$$

From the attention scores, attention weights are created via an alignment function, and are used to produce the context vectors $\boldsymbol{c}^{(1)}$ and $\boldsymbol{c}^{(2)}$.

While coarse-grained co-attention mechanisms use a compact representation of one input to use as a query when

calculating attention for another input, fine-grained co-attention considers every element of each input individually when calculating attention scores. In this case, the query becomes a matrix. An example of fine-grained co-attention is **parallel co-attention** [32]. Similarly to interactive co-attention, parallel co-attention calculates attention on the two feature matrices at the same time, as shown in Fig. 6. We start by evaluating the keys matrices $\boldsymbol{K}^{(1)} \in \mathbb{R}^{d_k^{(1)} \times n_f^{(1)}}$ and $\boldsymbol{K}^{(2)} \in \mathbb{R}^{d_k^{(2)} \times n_f^{(2)}}$ that are obtained by linearly transforming the feature matrices $\boldsymbol{F}^{(1)}$ and $\boldsymbol{F}^{(2)}$, where $d_k^{(1)}$ and $d_k^{(2)}$ are prespecified dimensions of the keys. The idea is to use the keys matrix from one input as the query for calculating attention on the other input. However, since $\boldsymbol{K}^{(1)}$ and $\boldsymbol{K}^{(2)}$ have completely different dimensions, an affinity matrix $\boldsymbol{A} \in \mathbb{R}^{n_f^{(1)} \times n_f^{(2)}}$ is calculated that is used to essentially translate one keys matrix to the space of the other keys. In [32], $\boldsymbol{A}$ is calculated as shown in (12).

$$\underset{n_f^{(1)} \times n_f^{(2)}}{\boldsymbol{A}} = \text{act}(\underset{n_f^{(1)} \times d_k^{(1)}}{\boldsymbol{K}^{(1)^T}} \times \underset{d_k^{(1)} \times d_k^{(2)}}{\boldsymbol{W}_A} \times \underset{d_k^{(2)} \times n_f^{(2)}}{\boldsymbol{K}^{(2)}}), \quad (12)$$

where $\boldsymbol{W}_A \in \mathbb{R}^{d_k^{(1)} \times d_k^{(2)}}$ is a trainable weights matrix and act() is an activation function for which the tanh() function is used in [32]. [35] proposes a different way of calculating this matrix, i.e., one can use (13) to calculate each individual element $A_{i,j}$ of the matrix $\boldsymbol{A}$.

$$\underset{1 \times 1}{A_{i,j}} = \underset{1 \times 3d_k}{\boldsymbol{w}_A^T} \times \text{concat}(\underset{d_k \times 1}{\boldsymbol{k}_i^{(1)}}, \underset{d_k \times 1}{\boldsymbol{k}_j^{(2)}}, \underset{d_k \times 1}{\boldsymbol{k}_i^{(1)}} \circ \underset{d_k \times 1}{\boldsymbol{k}_j^{(2)}}), \quad (13)$$

where $\boldsymbol{w}_A \in \mathbb{R}^{3d_k}$ denotes a trainable vector of weights, concat() denotes vector concatenation, and $\circ$ denotes element-wise multiplication, also known as the Hadamard product. Note that the keys of each keys matrix in this case must have the same dimension d_k for the element-wise multiplication to work. The affinity matrix can be interpreted as a similarity matrix for the columns of the two keys matrices, and helps translate, for example, image keys to the same space as the keys of the words in a sentence, and vice versa. The vectors of attention scores $\boldsymbol{e}^{(1)}$ and $\boldsymbol{e}^{(2)}$ can be calculated using an altered version of the additive score function as presented in (14) and (15). The previous attention score examples in this survey all used a score function to calculate each attention score for each value vector individually. However, (14) and (15) are used to calculate the complete vector of all attention scores. Essentially, the attention scores are calculated in an aggregated form.

$$\underset{1 \times n_f^{(1)}}{\boldsymbol{e}^{(1)}} = \underset{1 \times d_w}{\boldsymbol{w}_1} \times \text{act}(\underset{d_w \times d_k^{(2)}}{\boldsymbol{W}_2} \times \underset{d_k^{(2)} \times n_f^{(2)}}{\boldsymbol{K}^{(2)}} \times \underset{n_f^{(2)} \times n_f^{(1)}}{\boldsymbol{A}^T} + \underset{d_w \times d_k^{(1)}}{\boldsymbol{W}_1} \times \underset{d_k^{(1)} \times n_f^{(1)}}{\boldsymbol{K}^{(1)}}); \quad (14)$$

$$\underset{1 \times n_f^{(2)}}{\boldsymbol{e}^{(2)}} = \underset{1 \times d_w}{\boldsymbol{w}_2} \times \text{act}(\underset{d_w \times d_k^{(1)}}{\boldsymbol{W}_1} \times \underset{d_k^{(1)} \times n_f^{(1)}}{\boldsymbol{K}^{(1)}} \times \underset{n_f^{(1)} \times n_f^{(2)}}{\boldsymbol{A}} + \underset{d_w \times d_k^{(2)}}{\boldsymbol{W}_2} \times \underset{d_k^{(2)} \times n_f^{(2)}}{\boldsymbol{K}^{(2)}}), \quad (15)$$

where $\boldsymbol{w}_1 \in \mathbb{R}^{d_w}$, $\boldsymbol{w}_2 \in \mathbb{R}^{d_w}$, $\boldsymbol{W}_1 \in \mathbb{R}^{d_w \times d_k^{(1)}}$, and $\boldsymbol{W}_2 \in \mathbb{R}^{d_w \times d_k^{(2)}}$ are trainable weight matrices, for which d_w is a prespecified dimension of the weight matrices. Note that tanh() is used in [32] for the activation function, and the feature matrices are used as the key matrices. In that case, the affinity matrix $\boldsymbol{A}$ can be seen as a translator between feature spaces. As mentioned before, the affinity matrix is essentially a similarity matrix for the key vectors of the two inputs. In [33], this fact is used to propose a different way of determining attention scores. Namely, one could take the maximum similarity value in a row or column as the attention score, as shown in (16).

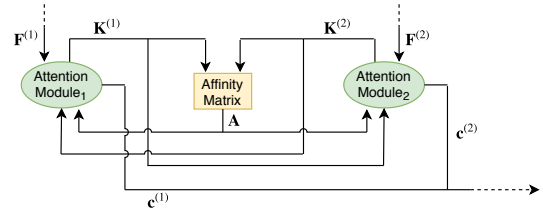

Fig. 6. An illustration of parallel co-attention.

$$\underset{1 \times 1}{e_i^{(1)}} = \max_{j=1,\ldots,n_f^{(2)}} \underset{1 \times 1}{A_{i,j}}, \qquad \underset{1 \times 1}{e_j^{(2)}} = \max_{i=1,\ldots,n_f^{(1)}} \underset{1 \times 1}{A_{i,j}}. \quad (16)$$

Next, the attention scores are used to calculate attention weights using an alignment function, so that two context vectors $\boldsymbol{c}^{(1)}$ and $\boldsymbol{c}^{(2)}$ can be derived as weighted averages of the value vectors that are obtained from linearly transforming the features. For the alignment function, [32] proposes to use a softmax function, and the value vectors are simply set equal to the feature vectors. The resulting context vectors can be either concatenated or added together.

Finally, coarse-grained and fine-grained co-attention can be combined to create an even more complex co-attention mechanism. [33] proposes the **multi-grained co-attention** mechanism that calculates both coarse-grained and fine-grained co-attention for two inputs. Each mechanism produces one context vector per input. The four resulting context vectors are concatenated and used in the output model for prediction.

A mechanism separate from co-attention that still uses multiple inputs is the **rotatory attention** mechanism [36]. This technique is typically used in a text sentiment analysis setting where there are three inputs involved: the phrase for which the sentiment needs to be determined (target phrase), the text before the target phrase (left context), and the text after the target phrase (right context). The words in these three inputs are all encoded by the feature model, producing the following feature matrices: $\boldsymbol{F}^t = [\boldsymbol{f}_1^t, \ldots, \boldsymbol{f}_{n_f^t}^t] \in \mathbb{R}^{d_f^t \times n_f^t}$, $\boldsymbol{F}^l = [\boldsymbol{f}_1^l, \ldots, \boldsymbol{f}_{n_f^l}^l] \in \mathbb{R}^{d_f^l \times n_f^l}$, and $\boldsymbol{F}^r = [\boldsymbol{f}_1^r, \ldots, \boldsymbol{f}_{n_f^r}^r] \in \mathbb{R}^{d_f^r \times n_f^r}$, for the target phrase words, left context words, and right context words, respectively, where d_f^t, d_f^l, and d_f^r represent the dimensions of the feature vectors for the corresponding inputs, and n_f^t, n_f^l, and n_f^r represent the number of feature vectors for the corresponding inputs. The feature model used in [36] consists of word embeddings and separate Bi-LSTM models for the target phrase, the left context, and the right context. This means that the feature vectors are in fact the hidden state vectors obtained from the Bi-LSTM models. Using these features, the idea is to extract a single vector $\boldsymbol{r}$ from the inputs such that a softmax layer can be used for classification. As such, we are now faced with two challenges: how to represent the inputs as a single vector, and how to incorporate the information from the left and right context into that vector. [36] proposes to use the rotatory attention mechanism for this purpose.